HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA ĐA PHƯƠNG TIỆN

BỘ MÔN CHUYÊN ĐỀ PHÁT TRIỂN ỨNG DỤNG ĐA PHƯƠNG TIỆN

BÁO CÁO
PHÁT TRIỂN ỨNG DỤNG IOT

Giảng viên hướng dẫn	**: Nguyễn Quốc Uy**
Lớp	**: B22PTDPT - 02**
Nhóm môn học	**: 01**
	Vũ Đình Tú - B22DCPT244

Hà Nội – 2026

LỜI CẢM ƠN

Trong suốt quá trình thực hiện và hoàn thành đồ án này, em đã nhận được rất nhiều sự giúp đỡ, hướng dẫn và động viên quý báu từ thầy cô, gia đình và bạn bè.

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc tới giảng viên hướng dẫn Thầy Nguyễn Quốc Uy, người đã tận tình chỉ bảo, định hướng và hỗ trợ em trong suốt quá trình thực hiện đề tài. Những góp ý chuyên môn quý báu của thầy không chỉ giúp em hoàn thiện đồ án mà còn giúp em nâng cao kiến thức và tư duy trong lĩnh vực công nghệ.

Em cũng xin chân thành cảm ơn các thầy cô trong khoa đã truyền đạt cho em những kiến thức nền tảng và chuyên sâu trong suốt quá trình học tập tại trường. Đây chính là hành trang quan trọng giúp em có thể thực hiện và hoàn thành đề tài này.

Bên cạnh đó, em xin gửi lời cảm ơn đến gia đình và bạn bè đã luôn bên cạnh, động viên và hỗ trợ em cả về tinh thần lẫn vật chất trong suốt thời gian học tập và thực hiện đồ án.

Mặc dù đã cố gắng hết sức, nhưng do hạn chế về thời gian và kinh nghiệm, đồ án không tránh khỏi những thiếu sót. Em rất mong nhận được sự thông cảm và những ý kiến đóng góp quý báu từ thầy cô để đề tài được hoàn thiện hơn.

Em xin chân thành cảm ơn!

MỤC LỤC

Lời cảm ơn 2

Mục lục..... 3

Mục lục hình ảnh.....7

CHƯƠNG 1: MỞ ĐẦU..... 8

1. Bối cảnh và xu hướng công nghệ 8

a) Xu hướng phát triển IoT trên thế giới 8

b) Xu hướng công nghệ Web hiện đại 8

c) Xu hướng giao tiếp thời gian thực 9

d) Thực tiễn tại Việt Nam 9

e) Định hướng của đề tài 9

2. Vấn đề đặt ra 10

a) Mục tiêu tổng quát 10

b) Mục tiêu cụ thể 10

1. Thiết kế và triển khai phần cứng 10
2. Xây dựng firmware cho ESP32 10
3. Phát triển hệ thống Backend 10
4. Phát triển Web Dashboard (Frontend) 11
5. Đồng bộ hóa trạng thái hệ thống 11
6. Lưu trữ và phân tích dữ liệu 11
7. Đánh giá và kiểm thử hệ thống 11

c) Mục tiêu nâng cao 11

d) Kết quả mong đợi 12

3. Mục tiêu chi tiết 12

a) Đối tượng nghiên cứu 12

1. Vi điều khiển và hệ thống nhúng 12
2. Cảm biến môi trường 12
3. Giao thức truyền thông IoT 12
4. Hệ thống Backend 13

5. Công nghệ Web Frontend 13
6. Cơ chế đồng bộ trạng thái 13
b) Phạm vi nghiên cứu 13
1. Phạm vi không gian 13
2. Phạm vi chức năng 13
c) Giới hạn của đề tài 14
d) Ý nghĩa của phạm vi nghiên cứu 14
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT 15
1. Kiến trúc IoT 15
1. Perception Layer (Tầng cảm nhận) 15
2. Network Layer (Tầng mạng) 15
3. Processing Layer (Tầng xử lý) 15
4. Application Layer (Tầng ứng dụng) 15
2. Giao thức MQTT 15
a) Mô hình Publish/Subscribe 16
b) Ưu điểm của MQTT 16
c) Các mức QoS (Quality of Service) 16
d) Vai trò của MQTT trong hệ thống đề tài 17
3. ESP32 17
a) Đặc điểm kỹ thuật 17
b) Ưu điểm của ESP32 17
c) Vai trò của ESP32 trong hệ thống 18
d) Lý do lựa chọn ESP32 18
4. Cảm biến 19
a) Cảm biến DHT11 19
b) Cảm biến BH1750 19
c) So sánh và lựa chọn 20
5. Công nghệ Web 20
a) Next.js 21
b) Socket.io 21

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ 22

1. Phân tích nghiệp vụ 22

a) Người dùng 22

b) Luồng hoạt động của hệ thống 23

2. Thiết kế kiến trúc chi tiết 24

a) Luồng dữ liệu (Data Flow) 24

b) Luồng điều khiển (Control Flow) 25

3. Thiết kế Database 26

4. Thiết kế UI/UX 27

● Tổng quan Dashboard 27

● Khu vực hiển thị dữ liệu cảm biến 28

● Khu vực điều khiển thiết bị 28

● Biểu đồ dữ liệu (Chart) 29

● Action History - Trạng thái hệ thống 29

● Profile - Giao diện người dùng 30

CHƯƠNG 4: HIỆN THỰC 31

1. Firmware ESP32 (Phân tích sâu) 31

a) Vấn đề của phương pháp blocking (delay()) 31

b) Giải pháp Non-blocking với millis() 31

c) Ví dụ minh họa 32

d) Ưu điểm của phương pháp Non-blocking 32

e) Ứng dụng trong hệ thống 32

2. Xử lý MQTT 32

a) Cấu trúc topic MQTT 33

b) Luồng hoạt động MQTT 33

3. Backend Node.js 34

a) Chức năng chính của Backend 34

b) Luồng xử lý dữ liệu trong Backend 34

c) Xử lý điều khiển thiết bị 34

d) Ví dụ xử lý MQTT trong Node.js 35
e) Ưu điểm của kiến trúc Backend 35
4. Frontend 35
CHƯƠNG 5: THỬ NGHIỆM 37
1. Đo độ trễ 37
2. Kiểm tra độ ổn định khi mất kết nối 37
3. Đánh giá độ chính xác của cảm biến 38
4. Kiểm thử khả năng đồng bộ trạng thái 38
KẾT LUẬN 39
HƯỚNG PHÁT TRIỂN 42
PHỤ LỤC A: SOFTWARE REQUIREMENTS SPECIFICATION (SRS) 42
1. Giới thiệu 42
2. Mô tả tổng quan 43
3. Yêu cầu chức năng (Functional Requirements) 43
4. Yêu cầu phi chức năng (Non-functional Requirements) 44
5. Use Case Specification (Chi tiết) 44
6. Yêu cầu giao diện (UI Requirements) 45
7. Yêu cầu hệ thống (System Requirements) 45
8. Ràng buộc hệ thống 45
9. Tiêu chí chấp nhận (Acceptance Criteria) 46

MỤC LỤC HÌNH ẢNH

Hình 3.1: Sơ đồ Usecase 22
Hình 3.2: Sơ đồ Activity 23
Hình 3.3: Sơ đồ Sequence luồng dữ liệu 24
Hình 3.4: Sơ đồ ERD 27
Hình 3.5: Tổng quan giao diện hệ thống IoT 27
Hình 3.6: Khu vực hiển thị dữ liệu cảm biến 28
Hình 3.7: thiết bị on 28
Hình 3.8: thiết bị off 28
Hình 3.9: Biểu đồ dữ liệu theo thời gian 29
Hình 3.10: Trạng thái kết nối hệ thống 29
Hình 3.11: Giao diện profile 30

CHƯƠNG 1: MỞ ĐẦU

1. Bối cảnh và xu hướng công nghệ

Trong những năm gần đây, thế giới đang bước vào giai đoạn phát triển mạnh mẽ của cuộc cách mạng công nghiệp lần thứ tư (Industry 4.0), nơi các công nghệ số, tự động hóa và trí tuệ nhân tạo (AI) dần thay thế và nâng cao các phương thức vận hành truyền thống. Một trong những công nghệ cốt lõi đóng vai trò nền tảng cho sự chuyển đổi này chính là **Internet of Things (IoT)**.

IoT được hiểu là mạng lưới các thiết bị vật lý được tích hợp cảm biến, phần mềm và khả năng kết nối internet, cho phép chúng thu thập và trao đổi dữ liệu một cách tự động mà không cần sự can thiệp trực tiếp của con người. Nhờ đó, các hệ thống có thể giám sát, phân tích và đưa ra quyết định theo thời gian thực, góp phần nâng cao hiệu quả vận hành và tối ưu hóa tài nguyên.

a) Xu hướng phát triển IoT trên thế giới

Hiện nay, IoT đang được ứng dụng rộng rãi trong nhiều lĩnh vực:

- **Nhà thông minh (Smart Home):** Điều khiển thiết bị điện, giám sát môi trường sống từ xa.
- **Nông nghiệp thông minh (Smart Agriculture):** Theo dõi độ ẩm đất, ánh sáng, tự động tưới tiêu.
- **Công nghiệp (Industrial IoT):** Giám sát máy móc, dự đoán lỗi, tối ưu dây chuyền sản xuất.
- **Y tế (Smart Healthcare):** Theo dõi sức khỏe bệnh nhân theo thời gian thực.

Theo các báo cáo công nghệ gần đây, số lượng thiết bị IoT trên toàn cầu đang tăng theo cấp số nhân, kéo theo nhu cầu về các hệ thống xử lý dữ liệu thời gian thực, độ trễ thấp và độ tin cậy cao.

b) Xu hướng công nghệ Web hiện đại

Song song với sự phát triển của IoT, các công nghệ web cũng đã có những bước tiến vượt bậc. Các framework hiện đại như Next.js cho phép:

- Rendering phía server (SSR) giúp tối ưu hiệu năng
- Tăng tốc độ tải trang và cải thiện trải nghiệm người dùng

- Hỗ trợ xây dựng các ứng dụng realtime thông qua WebSocket

Việc kết hợp IoT với Web hiện đại giúp tạo ra các hệ thống giám sát trực quan, dễ sử dụng và có khả năng truy cập mọi lúc, mọi nơi.

c) Xu hướng giao tiếp thời gian thực

Một yếu tố quan trọng trong hệ thống IoT là khả năng truyền dữ liệu theo thời gian thực. Các giao thức nhẹ như MQTT đang trở thành tiêu chuẩn trong lĩnh vực này nhờ:

- Tiết kiệm băng thông
- Độ trễ thấp
- Hoạt động tốt trên thiết bị tài nguyên hạn chế

Bên cạnh đó, các công nghệ như Socket.io giúp đồng bộ dữ liệu giữa server và client gần như ngay lập tức, tạo trải nghiệm "real-time" mượt mà cho người dùng.

d) Thực tiễn tại Việt Nam

Tại Việt Nam, IoT đang dần được ứng dụng trong các lĩnh vực như:

- Nhà thông minh
- Giám sát môi trường (không khí, nhiệt độ, độ ẩm)
- Nông nghiệp công nghệ cao

Tuy nhiên, nhiều hệ thống hiện tại vẫn còn hạn chế:

- Thiếu tính đồng bộ dữ liệu
- Giao diện chưa thân thiện
- Độ trễ cao hoặc không ổn định

e) Định hướng của đề tài

Xuất phát từ các xu hướng trên, đề tài hướng tới việc xây dựng một hệ thống IoT hoàn chỉnh với các đặc điểm:

- Giám sát môi trường theo thời gian thực
- Điều khiển thiết bị từ xa thông qua web
- Đồng bộ trạng thái chính xác giữa phần cứng và giao diện
- Áp dụng các công nghệ hiện đại như ESP32, MQTT, Next.js và Socket.io

Hệ thống không chỉ mang tính ứng dụng thực tiễn mà còn thể hiện sự kết hợp giữa hệ thống nhúng và phát triển web hiện đại – một xu hướng tất yếu trong tương lai.

2. Vấn đề đặt ra

Mục tiêu của đề tài không chỉ dừng lại ở việc xây dựng một hệ thống IoT cơ bản, mà còn hướng tới việc phát triển một giải pháp hoàn chỉnh, có tính thực tiễn cao, khả năng mở rộng tốt và đáp ứng được các yêu cầu của một hệ thống hiện đại.

a) Mục tiêu tổng quát

Xây dựng một hệ thống giám sát môi trường và điều khiển thiết bị từ xa dựa trên nền tảng IoT, sử dụng vi điều khiển ESP32 kết hợp với các công nghệ web hiện đại, cho phép:

- Thu thập dữ liệu môi trường theo thời gian thực
- Hiển thị trực quan trên giao diện web
- Điều khiển thiết bị từ xa một cách ổn định và chính xác

b) Mục tiêu cụ thể

1. Thiết kế và triển khai phần cứng

- Lựa chọn và sử dụng vi điều khiển ESP32 làm trung tâm xử lý
- Kết nối và cấu hình các cảm biến:
    - DHT11: đo nhiệt độ và độ ẩm
    - BH1750: đo cường độ ánh sáng
- Thiết kế mạch điều khiển LED tương ứng với từng thông số môi trường
- Đảm bảo hệ thống phần cứng hoạt động ổn định, tiêu thụ điện năng thấp

2. Xây dựng firmware cho ESP32

- Lập trình đọc dữ liệu cảm biến theo chu kỳ
- Xử lý dữ liệu và đóng gói theo định dạng JSON
- Gửi dữ liệu lên MQTT Broker theo mô hình Publish/Subscribe
- Nhận lệnh điều khiển từ MQTT và thực thi ngay lập tức
- Áp dụng kỹ thuật lập trình **non-blocking** (millis()) để đảm bảo đa nhiệm
- Xây dựng cơ chế tự động reconnect khi mất WiFi hoặc MQTT

3. Phát triển hệ thống Backend

- Xây dựng server trung gian bằng Node.js
- Kết nối với MQTT Broker để nhận/gửi dữ liệu
- Xử lý dữ liệu cảm biến và lưu vào cơ sở dữ liệu MySQL
- Xây dựng API phục vụ frontend
- Tích hợp Socket.io để truyền dữ liệu realtime đến client
- Thiết kế cơ chế xử lý trạng thái "waiting → confirmed" nhằm đảm bảo đồng bộ

4. Phát triển Web Dashboard (Frontend)

- Xây dựng giao diện bằng Next.js
- Hiển thị dữ liệu môi trường theo thời gian thực
- Thiết kế biểu đồ trực quan (chart) cho dữ liệu lịch sử
- Tích hợp toggle switch để điều khiển thiết bị
- Thể hiện trạng thái thiết bị rõ ràng (Loading – Active – Offline)
- Thiết kế giao diện responsive, thân thiện với người dùng

5. Đồng bộ hóa trạng thái hệ thống (State Synchronization)

- Đảm bảo trạng thái hiển thị trên UI luôn khớp với trạng thái thực tế của thiết bị
- Xử lý các trường hợp:
    - Mất kết nối tạm thời
    - Lệnh gửi nhưng chưa được xác nhận
- Xây dựng cơ chế ACK (Acknowledgement) từ ESP32
- Sử dụng database làm "nguồn trạng thái chuẩn" (single source of truth)

6. Lưu trữ và phân tích dữ liệu

- Lưu trữ dữ liệu cảm biến theo dạng chuỗi thời gian (time-series)
- Hỗ trợ truy vấn dữ liệu theo khoảng thời gian
- Phục vụ hiển thị biểu đồ và phân tích xu hướng môi trường

7. Đánh giá và kiểm thử hệ thống

- Đo độ trễ truyền dữ liệu từ thiết bị đến UI
- Kiểm tra độ ổn định khi mất kết nối mạng
- Đánh giá độ chính xác của cảm biến
- Kiểm thử khả năng đồng bộ trạng thái

c) Mục tiêu nâng cao

Ngoài các mục tiêu chính, đề tài còn hướng tới các khả năng mở rộng:

- Tích hợp điều khiển bằng giọng nói (Google Assistant, Alexa)

- Triển khai hệ thống trên nền tảng Cloud (AWS, Azure)
- Áp dụng AI để phân tích và dự đoán dữ liệu môi trường
- Nâng cấp hệ thống bảo mật (JWT, HTTPS, phân quyền người dùng)

d) Kết quả mong đợi

Sau khi hoàn thành, hệ thống phải đạt được:

- Hoạt động ổn định, độ trễ thấp (< 1 giây)
- Giao diện trực quan, dễ sử dụng
- Dữ liệu hiển thị chính xác và đồng bộ
- Có khả năng mở rộng và ứng dụng thực tế

3. Mục tiêu chi tiết

Trong quá trình thực hiện đề tài, việc xác định rõ đối tượng và phạm vi nghiên cứu đóng vai trò quan trọng nhằm giới hạn bài toán, tập trung vào các thành phần cốt lõi và đảm bảo tính khả thi trong thời gian triển khai.

a) Đối tượng nghiên cứu

Đề tài tập trung nghiên cứu và triển khai các thành phần chính sau:

1. Vi điều khiển và hệ thống nhúng

- Nghiên cứu kiến trúc và cách hoạt động của vi điều khiển ESP32
- Khả năng xử lý đa nhiệm, kết nối WiFi và giao tiếp với các thiết bị ngoại vi
- Lập trình firmware để thu thập dữ liệu và xử lý tín hiệu từ cảm biến

2. Cảm biến môi trường

- Cảm biến DHT11: đo nhiệt độ và độ ẩm
- Cảm biến BH1750: đo cường độ ánh sáng
- Cách thức thu thập, xử lý và hiệu chỉnh dữ liệu từ cảm biến

3. Giao thức truyền thông IoT

- Giao thức MQTT với mô hình Publish/Subscribe
- Cách tổ chức topic, truyền nhận dữ liệu và đảm bảo độ tin cậy
- Cơ chế ACK (Acknowledgement) để xác nhận trạng thái thiết bị

4. Hệ thống Backend

- Xây dựng server trung gian bằng Node.js
- Xử lý dữ liệu từ MQTT và chuyển tiếp đến frontend
- Thiết kế API và quản lý logic hệ thống
- Kết nối và thao tác với cơ sở dữ liệu MySQL

5. Công nghệ Web Frontend

- Sử dụng Next.js để xây dựng giao diện người dùng
- Hiển thị dữ liệu thời gian thực thông qua Socket.io
- Thiết kế dashboard trực quan, dễ sử dụng

6. Cơ chế đồng bộ trạng thái (State Synchronization)

- Nghiên cứu cách đảm bảo tính nhất quán dữ liệu giữa:
    - Thiết bị (ESP32)
    - Server (Backend)
    - Giao diện (Frontend)
- Xử lý các trường hợp lệch trạng thái hoặc mất kết nối

b) Phạm vi nghiên cứu

Để đảm bảo tính khả thi và phù hợp với thời gian thực hiện đồ án, đề tài được giới hạn trong các phạm vi sau:

1. Phạm vi không gian

- Hệ thống hoạt động trong môi trường:
    - Gia đình (Smart Home)
    - Phòng thí nghiệm hoặc mô hình demo
- Kết nối mạng thông qua WiFi cục bộ hoặc internet cơ bản

2. Phạm vi chức năng

Hệ thống tập trung vào các chức năng chính:

- Giám sát nhiệt độ, độ ẩm và ánh sáng
- Điều khiển thiết bị đơn giản (LED)
- Hiển thị dữ liệu thời gian thực
- Lưu trữ và truy vấn dữ liệu lịch sử
- Đồng bộ trạng thái giữa thiết bị và giao diện

Không bao gồm:

- Điều khiển thiết bị công suất lớn (máy lạnh, động cơ công nghiệp)

- Xử lý dữ liệu phức tạp bằng AI (chỉ ở mức đề xuất)

3. Phạm vi công nghệ

- Phần cứng: ESP32, DHT11, BH1750, LED
- Giao thức: MQTT
- Backend: Node.js
- Frontend: Next.js + Socket.io
- Database: MySQL

Không sử dụng:

- Các nền tảng IoT thương mại (Firebase IoT, AWS IoT Core) trong phiên bản chính
- Hệ thống cloud phức tạp (chỉ đề xuất mở rộng)

4. Phạm vi thời gian thực hiện

- Thời gian triển khai trong khuôn khổ đồ án tốt nghiệp
- Tập trung vào xây dựng hệ thống hoàn chỉnh, ổn định và có thể demo thực tế

c) Giới hạn của đề tài

Mặc dù hệ thống đã đáp ứng được các mục tiêu đề ra, vẫn tồn tại một số giới hạn:

- Cảm biến DHT11 có độ chính xác chưa cao
- Phụ thuộc vào kết nối mạng (WiFi/MQTT)
- Chưa tối ưu cho quy mô lớn (hàng trăm thiết bị)
- Chưa triển khai bảo mật nâng cao (OAuth2, Zero Trust,...)

d) Ý nghĩa của phạm vi nghiên cứu

Việc giới hạn phạm vi giúp:

- Tập trung vào các vấn đề cốt lõi của hệ thống IoT
- Đảm bảo tiến độ thực hiện đồ án
- Tạo nền tảng để phát triển mở rộng trong tương lai

Đồng thời, đề tài vẫn giữ được tính thực tiễn cao và có thể ứng dụng trực tiếp trong các mô hình giám sát môi trường quy mô nhỏ và vừa.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

1. Kiến trúc IoT

Một hệ thống Internet of Things (IoT) điển hình được thiết kế theo mô hình phân tầng nhằm đảm bảo khả năng thu thập, truyền tải, xử lý và hiển thị dữ liệu một cách hiệu quả. Trong đề tài này, kiến trúc hệ thống được chia thành **4 tầng chính**, mỗi tầng đảm nhận một vai trò riêng biệt:

1. Perception Layer (Tầng cảm nhận)

Đây là tầng thấp nhất của hệ thống, có nhiệm vụ thu thập dữ liệu từ môi trường thực tế thông qua các cảm biến. Trong đề tài, tầng này bao gồm các cảm biến như **DHT11** (đo nhiệt độ, độ ẩm) và **BH1750** (đo cường độ ánh sáng), được kết nối với vi điều khiển ESP32. Dữ liệu sau khi thu thập sẽ được chuyển đổi thành tín hiệu số để xử lý.

2. Network Layer (Tầng mạng)

Tầng mạng chịu trách nhiệm truyền dữ liệu từ thiết bị phần cứng đến hệ thống xử lý. ESP32 sử dụng kết nối WiFi để gửi dữ liệu thông qua giao thức MQTT. Giao thức này giúp đảm bảo việc truyền dữ liệu nhẹ, nhanh và phù hợp với các thiết bị có tài nguyên hạn chế.

3. Processing Layer (Tầng xử lý)

Đây là trung tâm xử lý của hệ thống, nơi tiếp nhận dữ liệu từ tầng mạng và thực hiện các chức năng như xử lý logic, lưu trữ dữ liệu và điều phối hệ thống. Trong đề tài, tầng này được triển khai bằng backend Node.js, kết hợp với cơ sở dữ liệu MySQL để lưu trữ dữ liệu cảm biến và trạng thái thiết bị.

4. Application Layer (Tầng ứng dụng)

Tầng ứng dụng là nơi người dùng tương tác trực tiếp với hệ thống thông qua giao diện web. Trong đề tài, Web Dashboard được xây dựng bằng Next.js, cho phép hiển thị dữ liệu theo thời gian thực, biểu đồ lịch sử và cung cấp chức năng điều khiển thiết bị từ xa.

2. Giao thức MQTT

MQTT (Message Queuing Telemetry Transport) là một giao thức truyền thông nhẹ, được thiết kế dành riêng cho các hệ thống IoT, nơi các thiết bị có tài nguyên hạn chế và mạng truyền không ổn định. MQTT hoạt động theo mô hình **Publish/Subscribe**,

giúp tách biệt giữa bên gửi và bên nhận dữ liệu, từ đó tăng tính linh hoạt và khả năng mở rộng của hệ thống.

a) Mô hình Publish/Subscribe

Trong mô hình này, các thành phần không giao tiếp trực tiếp với nhau mà thông qua một trung gian gọi là **Broker**:

- **Publisher:** Là thiết bị gửi dữ liệu. Trong đề tài, ESP32 đóng vai trò Publisher, gửi dữ liệu cảm biến như nhiệt độ, độ ẩm và ánh sáng lên hệ thống.
- **Broker:** Là máy chủ trung gian (ví dụ: Mosquitto), có nhiệm vụ nhận dữ liệu từ Publisher và phân phối đến các Subscriber tương ứng dựa trên topic.
- **Subscriber:** Là thành phần nhận dữ liệu. Trong hệ thống này, Backend Node.js đóng vai trò Subscriber, nhận dữ liệu từ ESP32 để xử lý và lưu trữ.

Cách tiếp cận này giúp hệ thống không bị phụ thuộc trực tiếp giữa các thành phần, từ đó dễ mở rộng và bảo trì.

b) Ưu điểm của MQTT

MQTT được sử dụng phổ biến trong IoT nhờ các ưu điểm sau:

- **Băng thông thấp:** Header nhỏ, tối ưu cho mạng yếu
- **Độ trễ thấp:** Truyền dữ liệu nhanh, phù hợp thời gian thực
- **Nhẹ (Lightweight):** Phù hợp với vi điều khiển như ESP32
- **Hỗ trợ QoS:** Đảm bảo độ tin cậy trong truyền dữ liệu
- **Giao tiếp bất đồng bộ:** Không cần chờ phản hồi trực tiếp như HTTP

c) Các mức QoS (Quality of Service)

MQTT cung cấp 3 mức QoS để đảm bảo độ tin cậy khi truyền dữ liệu:

- **QoS 0 – At most once (Tối đa 1 lần):**
  Dữ liệu được gửi một lần duy nhất, không có cơ chế kiểm tra hay gửi lại. Phù hợp với dữ liệu không quan trọng.
- **QoS 1 – At least once (Ít nhất 1 lần):**
  Dữ liệu sẽ được gửi lại nếu chưa nhận được ACK từ phía nhận. Đảm bảo dữ liệu không bị mất nhưng có thể bị trùng lặp.
- **QoS 2 – Exactly once (Chính xác 1 lần):**
  Đảm bảo dữ liệu được nhận đúng một lần duy nhất thông qua cơ chế bắt tay nhiều bước. Tuy nhiên, mức này có độ trễ cao hơn và ít được sử dụng trong hệ thống đơn giản.

Trong đề tài này, **QoS 1** được lựa chọn nhằm cân bằng giữa độ tin cậy và hiệu năng.

d) Vai trò của MQTT trong hệ thống đề tài

MQTT đóng vai trò là kênh truyền thông chính giữa phần cứng và phần mềm:

- ESP32 gửi dữ liệu cảm biến → MQTT Broker
- Backend nhận dữ liệu → xử lý → lưu database
- Backend gửi lệnh điều khiển → MQTT → ESP32
- ESP32 phản hồi trạng thái (ACK)

Nhờ đó, hệ thống đạt được:

- Truyền dữ liệu nhanh và ổn định
- Tách biệt giữa các thành phần
- Dễ dàng mở rộng trong tương lai

3. ESP32

ESP32 là một vi điều khiển mạnh mẽ do Espressif phát triển, được sử dụng rộng rãi trong các hệ thống IoT nhờ khả năng tích hợp WiFi, Bluetooth và hiệu năng cao. Trong đề tài này, ESP32 đóng vai trò là **thiết bị trung tâm**, chịu trách nhiệm thu thập dữ liệu từ cảm biến và giao tiếp với hệ thống thông qua mạng.

a) Đặc điểm kỹ thuật

ESP32 sở hữu nhiều tính năng nổi bật, phù hợp với các ứng dụng IoT:

- **CPU dual-core 240MHz:**
  Cho phép xử lý song song nhiều tác vụ, đáp ứng tốt các yêu cầu thời gian thực.
- **Kết nối WiFi (802.11 b/g/n):**
  Hỗ trợ kết nối internet trực tiếp mà không cần module phụ, giúp giảm chi phí và đơn giản hóa hệ thống.
- **Bluetooth BLE:**
  Cho phép giao tiếp với các thiết bị khác như điện thoại hoặc cảm biến không dây.
- **GPIO linh hoạt:**
  Nhiều chân vào/ra hỗ trợ các giao thức như I2C, SPI, UART, giúp dễ dàng kết nối với nhiều loại cảm biến và thiết bị ngoại vi.

b) Ưu điểm của ESP32

ESP32 được lựa chọn trong đề tài nhờ các ưu điểm sau:

- **Giá thành thấp:**
  Phù hợp với các hệ thống IoT quy mô nhỏ và trung bình.
- **Tiết kiệm năng lượng:**
  Hỗ trợ nhiều chế độ tiết kiệm điện (deep sleep), phù hợp với thiết bị hoạt động lâu dài.
- **Hỗ trợ đa nhiệm (FreeRTOS):**
  Cho phép chạy nhiều tác vụ song song, ví dụ:
  - Đọc dữ liệu cảm biến
  - Gửi dữ liệu MQTT
  - Nhận lệnh điều khiển
- **Tích hợp nhiều chức năng:**
  Không cần thêm nhiều module ngoài, giúp hệ thống gọn nhẹ và ổn định hơn.

c) Vai trò của ESP32 trong hệ thống

Trong hệ thống của đề tài, ESP32 đóng vai trò là cầu nối giữa môi trường thực và hệ thống phần mềm:

- Thu thập dữ liệu từ cảm biến (DHT11, BH1750)
- Xử lý và đóng gói dữ liệu dưới dạng JSON
- Gửi dữ liệu lên MQTT Broker
- Nhận lệnh điều khiển từ backend
- Điều khiển thiết bị (LED) theo yêu cầu
- Gửi phản hồi trạng thái (ACK) để đồng bộ hệ thống

d) Lý do lựa chọn ESP32

So với các vi điều khiển khác như Arduino Uno hoặc ESP8266, ESP32 có nhiều ưu điểm vượt trội:

- Hiệu năng cao hơn
- Có Bluetooth (ESP8266 không có)
- Hỗ trợ đa nhiệm tốt hơn
- Phù hợp với các hệ thống IoT hiện đại

Do đó, ESP32 là lựa chọn phù hợp để xây dựng hệ thống giám sát và điều khiển trong đề tài này.

4. Cảm biến

Trong đề tài, hệ thống sử dụng hai loại cảm biến chính là **DHT11** và **BH1750** để thu thập dữ liệu môi trường. Mỗi cảm biến đảm nhận một nhiệm vụ riêng và có những đặc điểm kỹ thuật phù hợp với mục tiêu của hệ thống.

a) Cảm biến DHT11

DHT11 là cảm biến phổ biến dùng để đo **nhiệt độ và độ ẩm không khí**, được sử dụng rộng rãi trong các hệ thống IoT cơ bản nhờ giá thành thấp và dễ sử dụng.

Đặc điểm:

- Đo nhiệt độ và độ ẩm trong môi trường
- Xuất dữ liệu dạng tín hiệu số
- Thời gian đọc dữ liệu khoảng **1 giây/lần**

Ưu điểm:

- Giá thành rẻ, dễ triển khai
- Giao tiếp đơn giản với ESP32
- Phù hợp cho các hệ thống thử nghiệm và học tập

Nhược điểm:

- **Sai số tương đối cao** (±2°C đối với nhiệt độ, ±5% đối với độ ẩm)
- Tốc độ phản hồi chậm
- Không phù hợp với các ứng dụng yêu cầu độ chính xác cao

Vai trò trong hệ thống:
DHT11 được sử dụng để thu thập dữ liệu nhiệt độ và độ ẩm, phục vụ hiển thị trên dashboard và làm cơ sở để điều khiển thiết bị (ví dụ: bật LED theo ngưỡng nhiệt độ).

b) Cảm biến BH1750

BH1750 là cảm biến ánh sáng kỹ thuật số, dùng để đo **cường độ ánh sáng (lux)** với độ chính xác cao.

Đặc điểm:

- Đo ánh sáng theo đơn vị lux
- Dải đo rộng (1 – 65535 lux)
- Giao tiếp thông qua chuẩn **I2C**

Ưu điểm:

- **Độ chính xác cao** hơn so với các cảm biến analog như LDR
- Dữ liệu ổn định, ít nhiễu
- Dễ tích hợp với ESP32 thông qua I2C

Nhược điểm:

- Giá cao hơn so với cảm biến ánh sáng cơ bản (LDR)
- Phụ thuộc vào giao tiếp I2C (cần cấu hình đúng)

Vai trò trong hệ thống:
BH1750 giúp đo cường độ ánh sáng môi trường, hỗ trợ hiển thị dữ liệu realtime và có thể dùng để điều khiển thiết bị (ví dụ: bật đèn khi trời tối).

c) So sánh và lựa chọn

Tiêu chí	DHT11	BH1750
Chức năng	Nhiệt độ, độ ẩm	Ánh sáng (Lux)
Độ chính xác	Trung bình	Cao
Giao tiếp	Digital (1-wire)	I2C
Giá thành	Rẻ	Trung bình

Việc kết hợp hai cảm biến này giúp hệ thống có thể giám sát toàn diện các yếu tố môi trường cơ bản, đồng thời đảm bảo cân bằng giữa chi phí và hiệu năng.

5. Công nghệ Web

Trong hệ thống, các công nghệ web hiện đại được sử dụng nhằm đảm bảo khả năng hiển thị dữ liệu theo thời gian thực, hiệu năng cao và trải nghiệm người dùng tốt. Hai công nghệ chính được sử dụng là **Next.js** và **Socket.io**.

a) Next.js

Next.js là một framework phát triển web dựa trên React, hỗ trợ xây dựng các ứng dụng web hiện đại với hiệu năng cao và khả năng tối ưu tốt.

Đặc điểm nổi bật:

- **SSR (Server-Side Rendering):**
  Cho phép render nội dung ngay trên server trước khi gửi về trình duyệt, giúp tăng tốc độ tải trang và cải thiện trải nghiệm người dùng.
- **SEO tốt:**
  Nhờ SSR, nội dung trang web dễ dàng được các công cụ tìm kiếm index, phù hợp cho các hệ thống cần hiển thị thông tin công khai.
- **Routing động (Dynamic Routing):**
  Hỗ trợ xây dựng các đường dẫn linh hoạt, dễ dàng mở rộng hệ thống khi có thêm nhiều thiết bị hoặc module.

Vai trò trong hệ thống:
Next.js được sử dụng để xây dựng Web Dashboard, hiển thị dữ liệu cảm biến theo thời gian thực và cung cấp giao diện điều khiển thiết bị cho người dùng.

b) Socket.io

Socket.io là thư viện hỗ trợ giao tiếp hai chiều giữa client và server theo thời gian thực, dựa trên nền tảng WebSocket.

Đặc điểm nổi bật:

- **Giao tiếp realtime:**
  Cho phép dữ liệu được cập nhật ngay lập tức mà không cần tải lại trang.
- **Fallback polling:**
  Trong trường hợp WebSocket không khả dụng, Socket.io sẽ tự động chuyển sang cơ chế polling để đảm bảo kết nối luôn ổn định.
- **Kết nối hai chiều (bi-directional):**
  Server và client có thể gửi dữ liệu cho nhau liên tục.

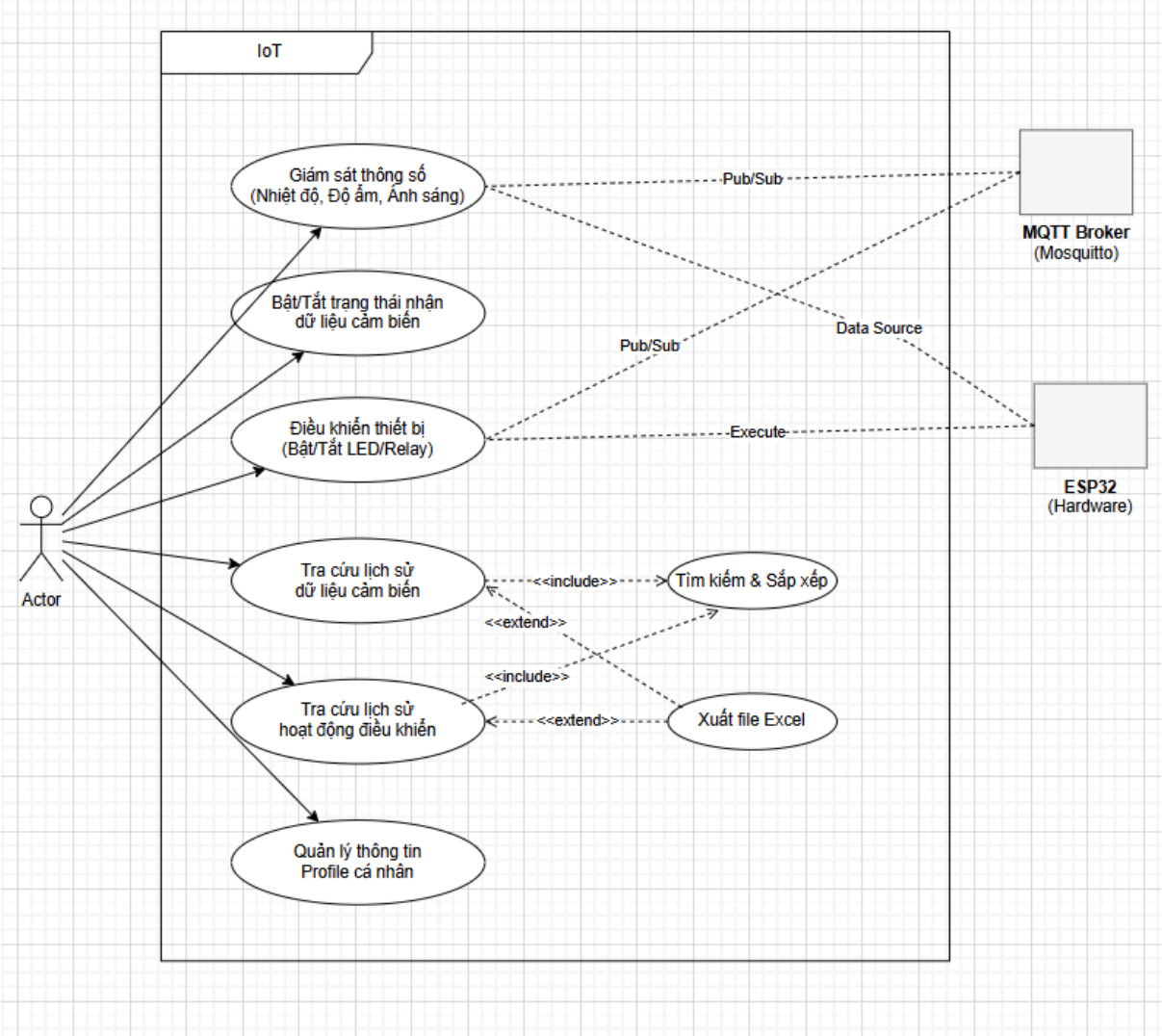
Vai trò trong hệ thống:
Socket.io được sử dụng để:

- Truyền dữ liệu cảm biến từ backend đến frontend theo thời gian thực
- Cập nhật trạng thái thiết bị ngay lập tức khi có thay đổi
- Đảm bảo UI luôn đồng bộ với trạng thái thực tế của hệ thống

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ

1. Phân tích nghiệp vụ

Hệ thống giám sát và điều khiển thiết bị IoT được xây dựng nhằm phục vụ nhóm người dùng chính với các quyền hạn và chức năng khác. Việc phân tích nghiệp vụ giúp xác định rõ cách hệ thống vận hành và luồng dữ liệu giữa các thành phần.



Hình 3.1: Sơ đồ Usecase

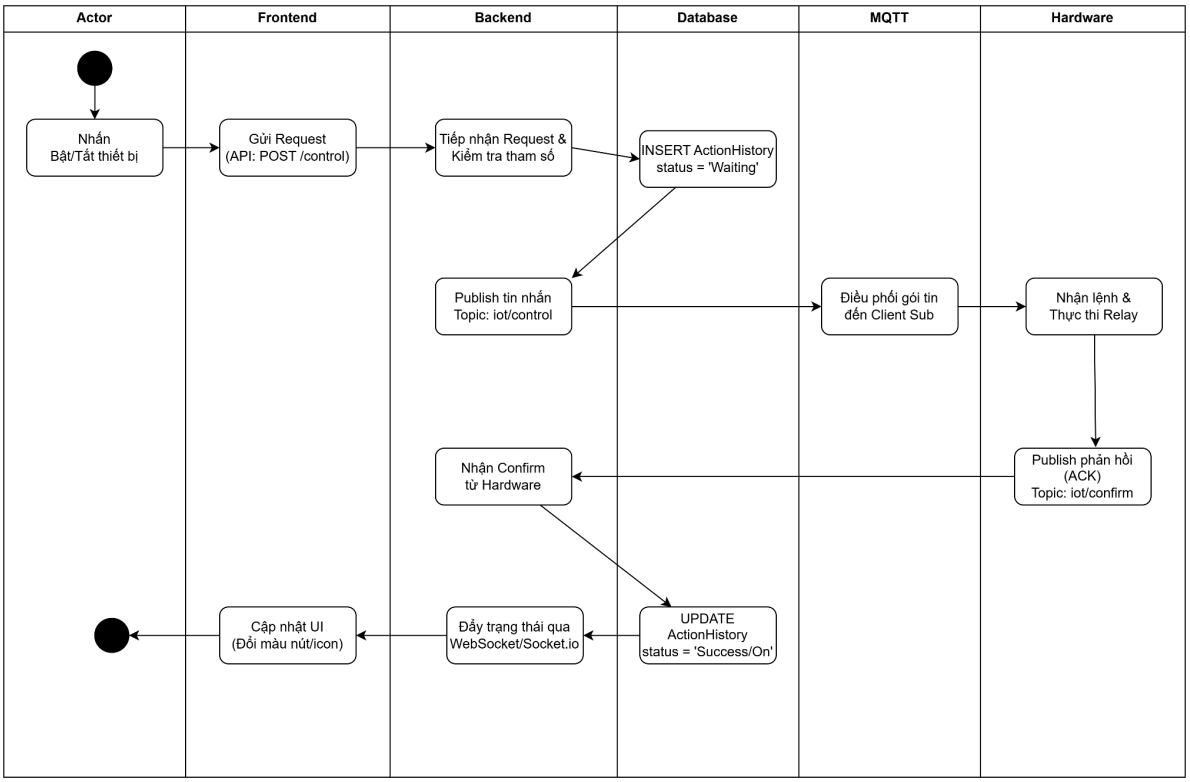
a) Người dùng

- **User:**
  Là người quản trị hệ thống, có toàn quyền điều khiển thiết bị và theo dõi hoạt động. Admin có thể:
  - Bật/tắt các thiết bị (LED)

- Theo dõi dữ liệu môi trường theo thời gian thực
- Xem lịch sử hoạt động và trạng thái hệ thống

b) Luồng hoạt động của hệ thống

Hệ thống hoạt động theo mô hình thu thập và xử lý dữ liệu theo thời gian thực, với các bước chính như sau:



Hình 3.2: Sơ đồ Activity

1. **ESP32 thu thập dữ liệu:**
   Các cảm biến (DHT11, BH1750) đo các thông số môi trường và gửi dữ liệu về ESP32.
2. **Gửi dữ liệu qua MQTT:**
   ESP32 đóng vai trò Publisher, gửi dữ liệu lên MQTT Broker theo các topic đã định nghĩa.
3. **Backend nhận và xử lý dữ liệu:**
   Backend (Node.js) đóng vai trò Subscriber, nhận dữ liệu từ MQTT, sau đó thực hiện xử lý logic cần thiết.
4. **Lưu trữ vào cơ sở dữ liệu:**
   Dữ liệu cảm biến được lưu vào hệ quản trị cơ sở dữ liệu MySQL để phục vụ việc truy vấn và phân tích sau này.

5. **Truyền dữ liệu realtime lên giao diện:**
   Backend sử dụng Socket.io để gửi dữ liệu đến frontend, giúp giao diện Web Dashboard cập nhật thông tin theo thời gian thực mà không cần tải lại trang.

c) Ý nghĩa của luồng nghiệp vụ

Luồng hoạt động trên giúp hệ thống đạt được:

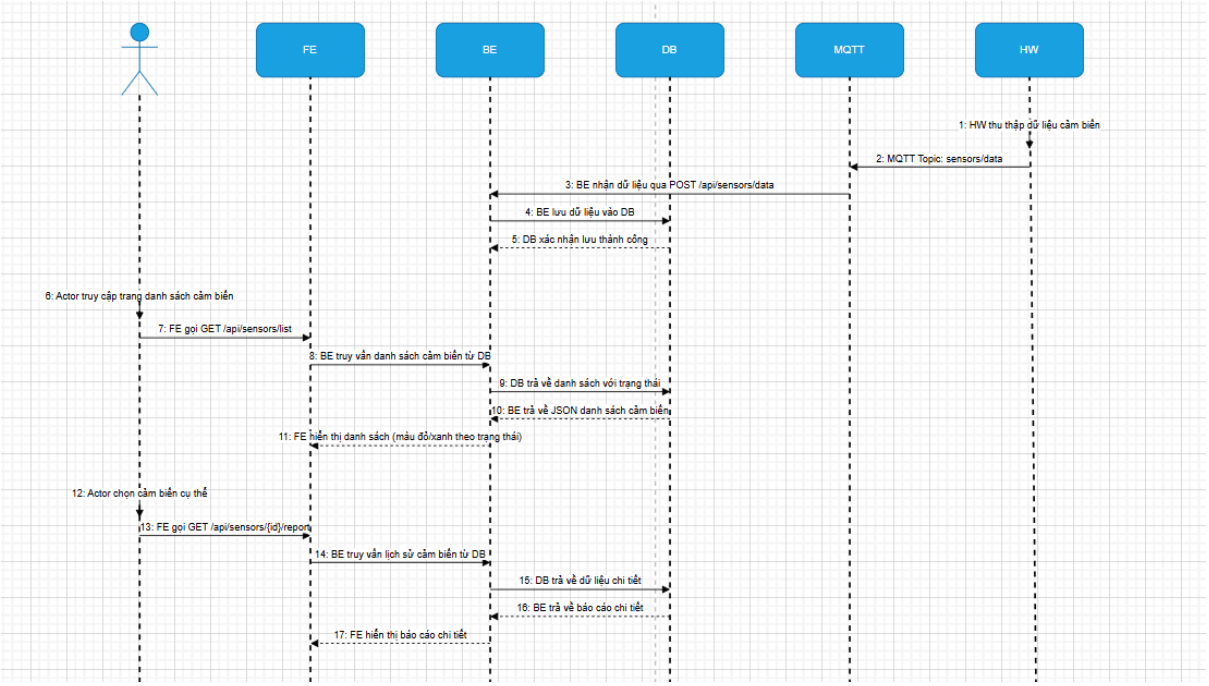
- Cập nhật dữ liệu nhanh chóng, gần như tức thời
- Tách biệt rõ ràng giữa phần cứng và phần mềm
- Dễ dàng mở rộng và nâng cấp hệ thống trong tương lai

2. Thiết kế kiến trúc chi tiết

Hệ thống được thiết kế theo mô hình phân tầng, trong đó các thành phần giao tiếp với nhau thông qua hai luồng chính: **luồng dữ liệu (data flow)** và **luồng điều khiển (control flow)**. Việc tách biệt hai luồng này giúp hệ thống hoạt động rõ ràng, dễ kiểm soát và đảm bảo tính đồng bộ.

a) Luồng dữ liệu (Data Flow)

Luồng dữ liệu mô tả quá trình thu thập và hiển thị thông tin môi trường theo thời gian thực:



Hình 3.3: Sơ đồ Sequence luồng dữ liệu

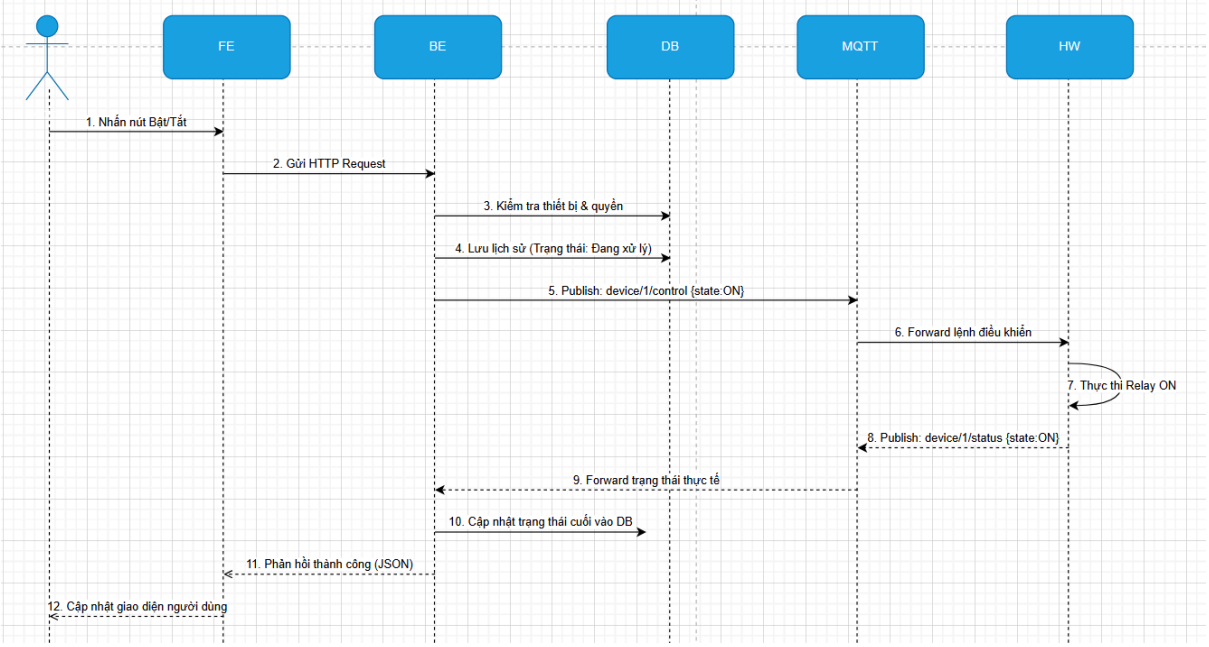
Sensor → ESP32 → MQTT → Backend → Database → Frontend

Chi tiết từng bước:

1. **Sensor (Cảm biến):**
   Các cảm biến như DHT11 và BH1750 thu thập dữ liệu môi trường (nhiệt độ, độ ẩm, ánh sáng).
2. **ESP32:**
   Nhận dữ liệu từ cảm biến, xử lý và đóng gói dưới dạng JSON, sau đó gửi lên MQTT Broker.
3. **MQTT Broker:**
   Đóng vai trò trung gian, nhận dữ liệu từ ESP32 và chuyển tiếp đến Backend.
4. **Backend (Node.js):**
   Nhận dữ liệu từ MQTT, xử lý và lưu vào cơ sở dữ liệu.
5. **Database (MySQL):**
   Lưu trữ dữ liệu cảm biến dưới dạng chuỗi thời gian để phục vụ truy vấn và phân tích.
6. **Frontend (Web Dashboard):**
   Nhận dữ liệu từ Backend thông qua Socket.io và hiển thị lên giao diện theo thời gian thực.

b) Luồng điều khiển (Control Flow)

Luồng điều khiển mô tả quá trình người dùng gửi lệnh và thiết bị thực thi:



Hình 3.3: Sơ đồ Sequence điều khiển

User → Frontend → Backend → MQTT → ESP32 → ACK → Backend → UI

Chi tiết từng bước:

1. **User (Người dùng):**
   Thực hiện thao tác điều khiển trên giao diện (ví dụ: bật/tắt LED).
2. **Frontend:**
   Gửi yêu cầu điều khiển đến Backend thông qua API hoặc WebSocket.
3. **Backend:**
   Xử lý yêu cầu và publish lệnh điều khiển lên MQTT Broker.
4. **MQTT Broker:**
   Chuyển tiếp lệnh đến ESP32 theo topic tương ứng.
5. **ESP32:**
   Nhận lệnh và thực thi (bật/tắt LED), sau đó gửi phản hồi (ACK).
6. **Backend:**
   Nhận ACK từ ESP32, cập nhật trạng thái vào database và gửi thông tin lên frontend.
7. **UI (Frontend):**
   Cập nhật trạng thái thiết bị theo thời gian thực, đảm bảo đồng bộ với thực tế.

c) Ý nghĩa của thiết kế kiến trúc

Việc tách riêng hai luồng dữ liệu và điều khiển mang lại nhiều lợi ích:

- Đảm bảo hệ thống hoạt động rõ ràng, dễ theo dõi
- Tăng khả năng đồng bộ giữa thiết bị và giao diện
- Dễ dàng mở rộng khi thêm nhiều thiết bị hoặc người dùng
- Giảm thiểu lỗi do xung đột dữ liệu

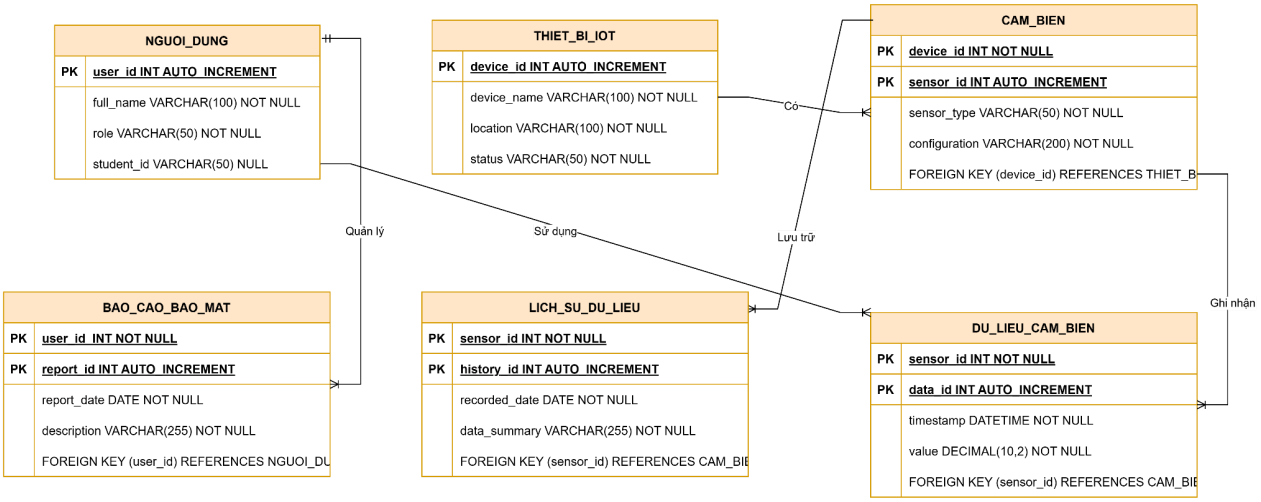
3. Thiết kế Database

Hệ thống sử dụng cơ sở dữ liệu quan hệ **MySQL** để lưu trữ và quản lý dữ liệu. Thiết kế database được xây dựng theo hướng tối ưu cho việc lưu trữ dữ liệu theo thời gian (time-series) và đảm bảo tính nhất quán giữa trạng thái thiết bị và giao diện người dùng.

a) Mục tiêu thiết kế

- Lưu trữ dữ liệu cảm biến theo thời gian thực
- Đảm bảo truy vấn nhanh cho biểu đồ và lịch sử
- Đồng bộ trạng thái thiết bị giữa phần cứng và frontend
- Ghi log các hoạt động điều khiển để phục vụ kiểm tra và debug

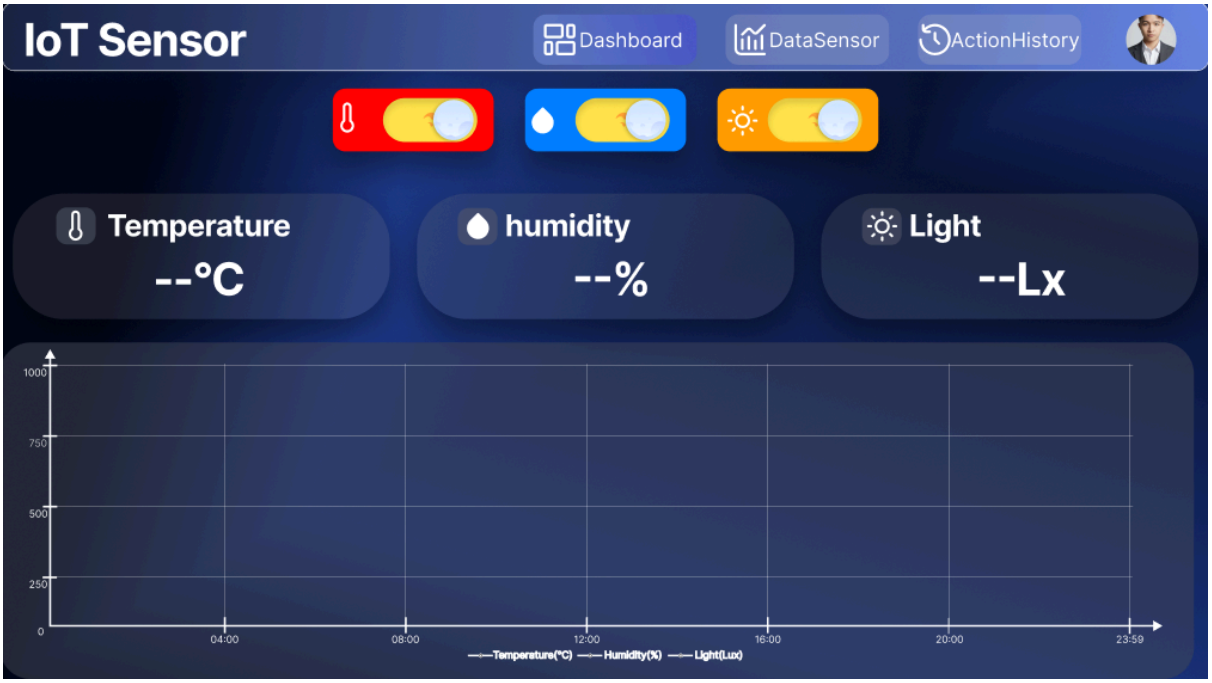
b) Các bảng chính trong hệ thống



Hình 3.4: Sơ đồ ERD

4. Thiết kế UI/UX

- Tổng quan Dashboard



Hình 3.5: Tổng quan giao diện hệ thống IoT

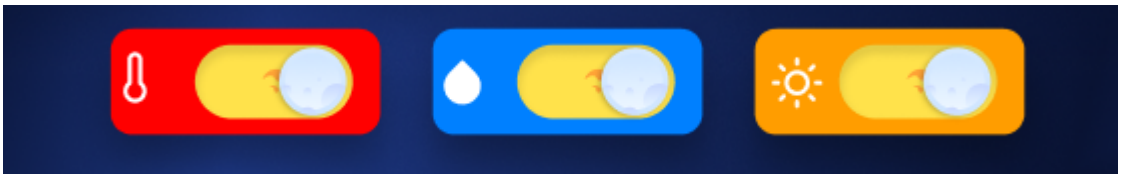
Giao diện tổng thể của hệ thống được thiết kế theo dạng dashboard, bao gồm các khu vực hiển thị dữ liệu cảm biến, biểu đồ và điều khiển thiết bị. Bố cục được sắp xếp khoa học giúp người dùng dễ dàng quan sát thông tin và thao tác trong thời gian thực. Thiết kế này phù hợp với các hệ thống IoT yêu cầu giám sát liên tục và phản hồi nhanh.

- Datasensor - hiển thị dữ liệu cảm biến

Hình 3.6: Khu vực hiển thị dữ liệu cảm biến

Khu vực này hiển thị các thông số môi trường như nhiệt độ, độ ẩm và cường độ ánh sáng. Dữ liệu được cập nhật liên tục từ hệ thống thông qua cơ chế realtime, giúp người dùng theo dõi trạng thái môi trường một cách trực quan và chính xác.

- Khu vực điều khiển thiết bị



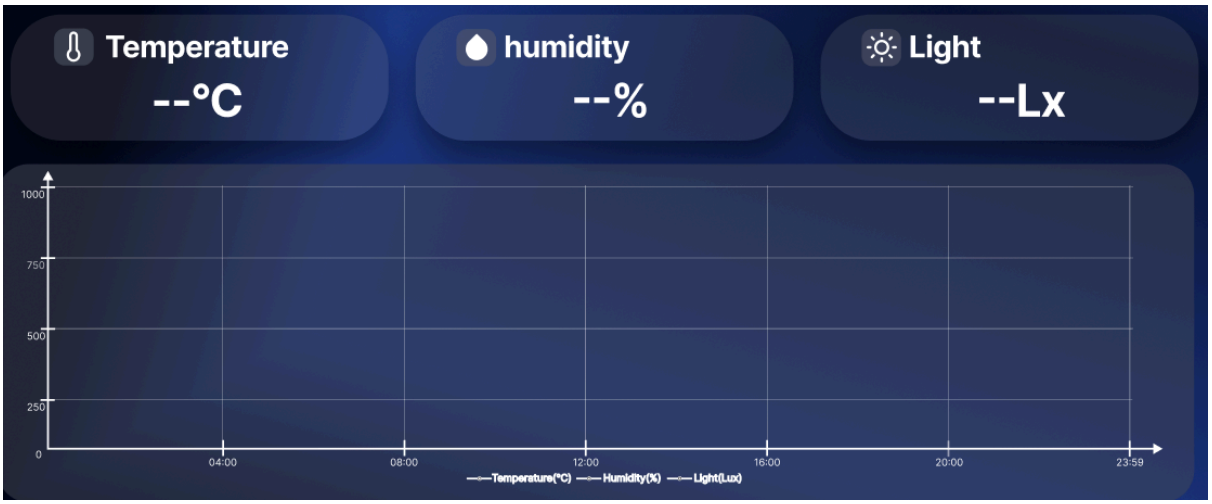
Hình 3.7: thiết bị on

Hình 3.8: thiết bị off

Khu vực điều khiển cho phép người dùng thực hiện các thao tác bật/tắt thiết bị thông qua các nút chức năng. Giao diện cung cấp trạng thái rõ ràng như ON,

OFF và WAITING. Khi người dùng thực hiện thao tác, hệ thống sẽ hiển thị trạng thái WAITING trước khi nhận phản hồi từ thiết bị, đảm bảo tính đồng bộ giữa giao diện và trạng thái thực tế.

- Biểu đồ dữ liệu (Chart)



Hình 3.9: Biểu đồ dữ liệu theo thời gian

Biểu đồ được sử dụng để hiển thị dữ liệu lịch sử của các thông số môi trường theo thời gian. Điều này giúp người dùng dễ dàng quan sát xu hướng biến đổi và đưa ra các đánh giá hoặc quyết định phù hợp. Giao diện biểu đồ được thiết kế trực quan, dễ đọc và hỗ trợ theo dõi dài hạn.

- Action History - Trạng thái hệ thống

ID	Tên cảm biến	Trạng thái	Mã lệnh	Thời gian
01	Cảm biến nhiệt độ	Wating	led_temp_on	17:20-16/01/2026
02	Cảm biến độ ẩm	Off	led_humi_off	17:16-16/01/2026
03	Cảm biến ánh sáng	Wating	led_bh_off	17:15-16/01/2026
04	Cảm biến ánh sáng	Off	led_bh_off	17:14-16/01/2026
05	Cảm biến độ ẩm	On	led_humi_on	17:13-16/01/2026
06	Cảm biến nhiệt độ	On	led_temp_on	17:11-16/01/2026
07	Cảm biến ánh sáng	On	led_bh_on	17:11-16/01/2026
08	Cảm biến nhiệt độ	Off	led_temp_off	17:10-16/01/2026
09	Cảm biến độ ẩm	Off	led_humi_off	17:10-16/01/2026
10	Cảm biến nhiệt độ	Off	led_temp_off	17:10-16/01/2026

Hình 3.10: Trạng thái kết nối hệ thống

Khu vực này hiển thị trạng thái hoạt động của hệ thống, bao gồm các trạng thái như Online, Offline hoặc đang kết nối lại. Thông tin này giúp người dùng nhận biết tình trạng kết nối giữa thiết bị và hệ thống, từ đó đảm bảo quá trình giám sát và điều khiển được thực hiện ổn định.

- Profile - Giao diện người dùng

Hình 3.11: Giao diện profile

CHƯƠNG 4: HIỆN THỰC (CHI TIẾT KỸ THUẬT)

1. Firmware ESP32 (Phân tích sâu)

Trong hệ thống IoT, firmware trên ESP32 đóng vai trò trung tâm trong việc thu thập dữ liệu, xử lý và giao tiếp với server. Để đảm bảo hệ thống hoạt động ổn định và không bị gián đoạn, đề tài sử dụng phương pháp lập trình **non-blocking** thay vì cách tiếp cận truyền thống sử dụng hàm delay().

a) Vấn đề của phương pháp blocking (delay())

Trong lập trình vi điều khiển, việc sử dụng delay() có thể gây ra nhiều hạn chế:

- Khi gọi delay(), vi điều khiển sẽ **dừng toàn bộ hoạt động** trong khoảng thời gian đó
- Không thể xử lý song song các tác vụ khác
- Dễ gây mất kết nối MQTT do không kịp xử lý dữ liệu đến
- Làm giảm khả năng phản hồi của hệ thống

b) Giải pháp Non-blocking với millis()

Để khắc phục nhược điểm trên, hệ thống sử dụng hàm millis() để kiểm soát thời gian mà **không làm gián đoạn chương trình**.

Nguyên lý hoạt động:

- millis() trả về thời gian (ms) kể từ khi ESP32 khởi động
- So sánh thời gian hiện tại với mốc thời gian trước đó
- Nếu đủ thời gian → thực hiện tác vụ

c) Ví dụ minh họa

```
void updateSensors() {
  if (!sensorsEnabled) return;
  unsigned long currentMillis = millis();
  if (currentMillis - lastSensorTime >= sensorInterval) {
    lastSensorTime = currentMillis;

    float h = readHumi ? dht.readHumidity() : 0.0;
    float t = readTemp ? dht.readTemperature() : 0.0;
    float lux = readLux ? lightMeter.readLightLevel() : 0.0;

    bool tempValid = readTemp ? !isnan(t) : true;
    bool humiValid = readHumi ? !isnan(h) : true;
    bool luxValid = readLux ? (lux >= 0) : true;

    if (!tempValid || !humiValid || !luxValid) {
      return; // Không in ra màn hình để tránh Lag quá mức
    }

    if (client.connected() && ESP.getFreeHeap() >= 8192) {
      char buffer[80];
      snprintf(buffer, sizeof(buffer),
        "{\"temp\":%.1f,\"humi\":%.1f,\"lux\":%.1f}", t, h, lux);
      client.publish(TOPIC_SENSORS, buffer);
    }
  }
}
```

d) Ưu điểm của phương pháp Non-blocking

- **Không làm treo hệ thống:**
  ESP32 vẫn hoạt động liên tục
- **Duy trì kết nối MQTT ổn định:**
  Có thể xử lý dữ liệu đến bất kỳ lúc nào
- **Hỗ trợ đa nhiệm:**
  Có thể thực hiện nhiều tác vụ song song (đọc cảm biến, nhận lệnh, gửi dữ liệu)
- **Tăng hiệu năng hệ thống:**
  Giảm độ trễ và tăng khả năng phản hồi

e) Ứng dụng trong hệ thống

Trong đề tài, phương pháp non-blocking được áp dụng cho:

- Đọc dữ liệu cảm biến định kỳ
- Gửi dữ liệu MQTT
- Nhận và xử lý lệnh điều khiển
- Điều khiển LED theo trạng thái

Nhờ đó, hệ thống đảm bảo hoạt động mượt mà và ổn định ngay cả khi có nhiều tác vụ diễn ra đồng thời.

2. Xử lý MQTT

Trong hệ thống IoT, MQTT được sử dụng làm giao thức truyền thông chính giữa ESP32 và Backend. Việc thiết kế topic hợp lý giúp hệ thống hoạt động rõ ràng, dễ mở rộng và dễ quản lý.

a) Cấu trúc topic MQTT

Hệ thống sử dụng hai nhóm topic chính:

- **Topic nhận lệnh (Subscribe):**

  ```
  #define TOPIC_CONTROL "esp32/control"
  ```

  ESP32 đăng ký (subscribe) vào topic này để nhận các lệnh điều khiển từ backend, ví dụ bật/tắt thiết bị.

- **Topic gửi dữ liệu (Publish):**

  ```
  #define TOPIC_SENSORS "esp32/sensors"
  ```

  ESP32 publish dữ liệu cảm biến lên topic này để backend nhận và xử lý.

b) Luồng hoạt động MQTT

Quá trình giao tiếp giữa ESP32 và backend thông qua MQTT diễn ra như sau:

1. ESP32 đọc dữ liệu từ cảm biến
2. Publish dữ liệu lên topic /device/data
3. Backend subscribe và nhận dữ liệu
4. Người dùng gửi lệnh điều khiển từ UI
5. Backend publish lệnh lên /device/control
6. ESP32 nhận lệnh và thực thi
7. ESP32 gửi phản hồi (ACK) về hệ thống

3. Backend Node.js

Backend trong hệ thống đóng vai trò là **trung tâm xử lý (Processing Layer)**, kết nối giữa phần cứng (ESP32) và giao diện người dùng (Frontend). Backend được xây dựng bằng Node.js nhằm tận dụng khả năng xử lý bất đồng bộ (asynchronous) và hỗ trợ realtime tốt.

a) Chức năng chính của Backend

Backend thực hiện ba chức năng cốt lõi:

- **Nhận dữ liệu từ MQTT:**
  Backend đóng vai trò là Subscriber, nhận dữ liệu cảm biến từ ESP32 thông qua MQTT Broker.
- **Ghi dữ liệu vào cơ sở dữ liệu (Database):**
  Dữ liệu sau khi nhận sẽ được xử lý và lưu vào MySQL để phục vụ truy vấn và hiển thị lịch sử.
- **Gửi dữ liệu realtime (Emit Socket):**
  Backend sử dụng Socket.io để gửi dữ liệu đến frontend ngay lập tức, đảm bảo giao diện luôn được cập nhật.

b) Luồng xử lý dữ liệu trong Backend

Quá trình xử lý dữ liệu diễn ra như sau:

1. Backend subscribe vào topic /esp32/data
2. Khi có dữ liệu từ ESP32 → nhận message
3. Parse dữ liệu JSON
4. Lưu vào database
5. Emit dữ liệu qua Socket.io đến frontend

c) Xử lý điều khiển thiết bị

Ngoài việc nhận dữ liệu, backend còn xử lý luồng điều khiển:

1. Nhận request từ frontend (bật/tắt thiết bị)
2. Ghi trạng thái tạm thời (waiting) vào database
3. Publish lệnh lên MQTT (/esp/control)
4. Chờ ESP32 gửi ACK
5. Cập nhật trạng thái chính thức (success)
6. Emit trạng thái mới lên frontend

d) Ví dụ xử lý MQTT trong Node.js

```
client.on("message", (topic, message) => {
  const data = JSON.parse(message.toString());

  // Lưu database
  saveToDB(data);

  // Gửi realtime
  io.emit("sensor-data", data);
});
```

e) Ưu điểm của kiến trúc Backend

- **Xử lý bất đồng bộ:**
  Node.js giúp xử lý nhiều request cùng lúc mà không bị block
- **Realtime mạnh mẽ:**
  Kết hợp MQTT + Socket.io giúp dữ liệu cập nhật ngay lập tức
- **Tách biệt rõ ràng:**
  Backend đóng vai trò trung gian giữa phần cứng và giao diện
- **Dễ mở rộng:**
  Có thể thêm nhiều thiết bị hoặc module mà không ảnh hưởng hệ thống

4. Frontend

Frontend của hệ thống được xây dựng nhằm cung cấp giao diện trực quan cho người dùng trong việc giám sát và điều khiển thiết bị IoT. Giao diện được phát triển bằng Next.js, kết hợp với Socket.io để đảm bảo khả năng cập nhật dữ liệu theo thời gian thực.

a) Chức năng chính của Frontend

Frontend đảm nhiệm các chức năng sau:

- **Hiển thị biểu đồ (Chart):**
  Dữ liệu cảm biến được trực quan hóa dưới dạng biểu đồ theo thời gian, giúp người dùng dễ dàng theo dõi xu hướng thay đổi của nhiệt độ, độ ẩm và ánh sáng.
- **Điều khiển thiết bị (Toggle Switch):**
  Cung cấp các nút bật/tắt (toggle) cho phép người dùng điều khiển thiết bị từ xa. Trạng thái của nút được cập nhật dựa trên phản hồi từ hệ thống.

- **Cập nhật realtime (Realtime Update):**
  Sử dụng Socket.io để nhận dữ liệu từ backend và cập nhật giao diện ngay lập tức mà không cần tải lại trang.

b) Luồng hoạt động của Frontend

Frontend hoạt động theo hai luồng chính:

Luồng hiển thị dữ liệu:

1. Nhận dữ liệu từ backend qua Socket.io
2. Cập nhật giá trị trên giao diện (card + chart)
3. Hiển thị dữ liệu mới theo thời gian thực

Luồng điều khiển thiết bị:

1. Người dùng thao tác bật/tắt thiết bị
2. Gửi request đến backend
3. UI chuyển sang trạng thái **WAITING**
4. Nhận phản hồi (ACK) từ backend
5. Cập nhật trạng thái ON/OFF

c) Thiết kế tương tác (Interaction)

Frontend được thiết kế với cơ chế phản hồi rõ ràng:

- Khi người dùng thao tác:
  - UI phản hồi ngay (đổi trạng thái)
  - Hiển thị trạng thái trung gian (WAITING)
- Khi có dữ liệu mới:
  - Tự động cập nhật mà không cần reload
  - Đảm bảo trải nghiệm mượt mà

d) Ưu điểm của Frontend

- **Trực quan và dễ sử dụng:**
  Giao diện dạng dashboard giúp người dùng nắm bắt thông tin nhanh
- **Realtime mạnh mẽ:**
  Dữ liệu được cập nhật liên tục
- **Phản hồi nhanh:**
  Giảm độ trễ trong thao tác điều khiển
- **Tính mở rộng cao:**
  Có thể thêm nhiều thiết bị hoặc module mới

CHƯƠNG 5: THỬ NGHIỆM

1. Đo độ trễ

Độ trễ là yếu tố quan trọng trong hệ thống IoT thời gian thực, phản ánh thời gian từ khi dữ liệu được gửi từ thiết bị đến khi hiển thị trên giao diện người dùng.

Phương pháp đo:

- Ghi lại thời điểm ESP32 gửi dữ liệu
- So sánh với thời điểm dữ liệu xuất hiện trên Web Dashboard

Kết quả:

- Độ trễ trung bình: **300 – 500 ms**

Đánh giá:

- Độ trễ dưới 1 giây được xem là phù hợp với hệ thống realtime
- Hệ thống đáp ứng tốt yêu cầu giám sát và điều khiển từ xa

2. Stress test

Thử nghiệm được thực hiện nhằm đánh giá khả năng xử lý của hệ thống khi có nhiều yêu cầu đồng thời.

Kịch bản thử nghiệm:

- Gửi khoảng **100 request/giây** từ nhiều client
- Bao gồm cả luồng dữ liệu cảm biến và luồng điều khiển

Kết quả:

- Hệ thống vẫn hoạt động ổn định
- Không xảy ra lỗi crash hoặc mất kết nối
- Backend xử lý tốt nhờ cơ chế bất đồng bộ của Node.js

Đánh giá:

- Hệ thống có khả năng chịu tải tốt trong phạm vi sử dụng thực tế
- Có thể mở rộng thêm nếu triển khai trên server mạnh hơn

3. Độ tin cậy

Độ tin cậy được đánh giá thông qua khả năng duy trì kết nối và xử lý khi xảy ra lỗi.

Các kịch bản thử nghiệm:

- Ngắt kết nối WiFi của ESP32
- Khởi động lại MQTT Broker
- Mất kết nối tạm thời giữa frontend và backend

Cơ chế xử lý:

- **Reconnect MQTT:**
  ESP32 tự động kết nối lại với MQTT Broker sau khi mất kết nối
- **Retry Logic:**
  Các lệnh điều khiển sẽ được gửi lại nếu chưa nhận được phản hồi (ACK)
- **Frontend Reconnect:**
  Socket.io tự động kết nối lại khi mất mạng

Kết quả:

- Hệ thống phục hồi sau khoảng 3–5 giây
- Không mất dữ liệu quan trọng
- Trạng thái thiết bị được đồng bộ lại chính xác

Đánh giá:

- Hệ thống có độ ổn định cao
- Đáp ứng tốt các tình huống lỗi trong thực tế

KẾT LUẬN

Sau quá trình nghiên cứu, thiết kế và triển khai, đề tài **"Xây dựng hệ thống giám sát môi trường và điều khiển thiết bị từ xa dựa trên nền tảng IoT (ESP32 & Next.js)"** đã hoàn thành các mục tiêu đề ra.

Hệ thống đã xây dựng thành công một mô hình IoT hoàn chỉnh, bao gồm phần cứng (ESP32 và cảm biến), backend xử lý dữ liệu và frontend hiển thị giao diện người dùng. Các thành phần được kết nối thông qua giao thức MQTT và cơ chế realtime, đảm bảo khả năng truyền tải dữ liệu nhanh chóng và chính xác.

Kết quả thử nghiệm cho thấy hệ thống đạt được các tiêu chí quan trọng:

- **Tính ổn định:**
  Hệ thống hoạt động liên tục, không xảy ra lỗi nghiêm trọng trong quá trình vận hành. Các cơ chế như reconnect MQTT và retry logic giúp hệ thống tự phục hồi khi xảy ra sự cố.
- **Hiệu năng tốt:**
  Độ trễ thấp (300–500ms) đáp ứng yêu cầu của hệ thống thời gian thực. Backend xử lý hiệu quả nhiều request đồng thời mà không bị quá tải.
- **Khả năng mở rộng:**
  Kiến trúc hệ thống được thiết kế theo mô hình phân tầng, dễ dàng mở rộng thêm thiết bị, người dùng hoặc tích hợp các công nghệ mới trong tương lai.
- **Tính đồng bộ cao:**
  Cơ chế đồng bộ trạng thái (WAITING → ACK) giúp đảm bảo sự nhất quán giữa giao diện người dùng và trạng thái thực tế của thiết bị.

HƯỚNG PHÁT TRIỂN

Trong tương lai, hệ thống có thể được mở rộng và nâng cấp nhằm tăng hiệu năng, tính thông minh và khả năng ứng dụng thực tế. Một số hướng phát triển cụ thể như sau:

1. Ứng dụng AI trong phân tích dữ liệu

Hệ thống có thể tích hợp các thuật toán Trí tuệ nhân tạo (AI) để phân tích dữ liệu môi trường thu thập được theo thời gian.

- Dự đoán xu hướng nhiệt độ, độ ẩm
- Phát hiện bất thường (anomaly detection)
- Đưa ra cảnh báo sớm cho người dùng

2. Phát triển ứng dụng Mobile

Xây dựng ứng dụng di động (Android/iOS) giúp người dùng:

- Theo dõi dữ liệu mọi lúc mọi nơi
- Điều khiển thiết bị tiện lợi hơn
- Nhận thông báo (push notification) khi có sự cố

3. Triển khai trên nền tảng Cloud

Hiện tại hệ thống hoạt động trong mạng cục bộ, trong tương lai có thể triển khai trên các nền tảng Cloud như:

- AWS
- Azure
- Vercel (cho frontend)

Lợi ích:

- Truy cập từ mọi nơi qua Internet
- Tăng khả năng mở rộng (scalability)
- Đảm bảo tính sẵn sàng cao (high availability)

4. Tăng cường bảo mật hệ thống

Để đảm bảo an toàn dữ liệu và hệ thống, cần bổ sung các cơ chế bảo mật:

- **JWT (JSON Web Token):**
  Xác thực và phân quyền người dùng

- **SSL/TLS:**
  Mã hóa dữ liệu truyền tải
- **Xác thực MQTT:**
  Bảo vệ broker khỏi truy cập trái phép

Kết luận

Các hướng phát triển trên giúp hệ thống tiến gần hơn đến một nền tảng IoT hoàn chỉnh, thông minh và có khả năng ứng dụng thực tế cao trong các lĩnh vực như nhà thông minh, nông nghiệp và giám sát môi trường.

PHỤ LỤC: SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

1. Giới thiệu

1.1 Mục đích

Tài liệu SRS này mô tả chi tiết các yêu cầu chức năng và phi chức năng của hệ thống giám sát môi trường và điều khiển thiết bị từ xa dựa trên IoT (ESP32 & Next.js). Tài liệu giúp định hướng phát triển, kiểm thử và bảo trì hệ thống.

1.2 Phạm vi hệ thống

Hệ thống cho phép:

- Thu thập dữ liệu môi trường (nhiệt độ, độ ẩm, ánh sáng)
- Hiển thị dữ liệu theo thời gian thực trên web
- Điều khiển thiết bị (LED) từ xa
- Lưu trữ và truy vấn dữ liệu lịch sử

1.3 Định nghĩa, từ viết tắt

- IoT: Internet of Things
- MQTT: Message Queuing Telemetry Transport
- UI: User Interface
- API: Application Programming Interface

2. Mô tả tổng quan

2.1 Góc nhìn sản phẩm

Hệ thống là một nền tảng IoT hoàn chỉnh gồm:

- Phần cứng (ESP32 + cảm biến)
- Backend (Node.js)
- Frontend (Next.js)
- Database (MySQL)

2.2 Người dùng hệ thống

- Admin: điều khiển thiết bị, xem log
- User: xem dữ liệu

2.3 Môi trường hoạt động

- Trình duyệt web (Chrome, Edge)
- ESP32 kết nối WiFi

3. Yêu cầu chức năng (Functional Requirements)

FR1: Thu thập dữ liệu

- ESP32 phải đọc dữ liệu từ cảm biến mỗi X giây
- Dữ liệu bao gồm: nhiệt độ, độ ẩm, ánh sáng

FR2: Truyền dữ liệu

- ESP32 publish dữ liệu qua MQTT
- Backend subscribe và xử lý dữ liệu

FR3: Hiển thị dữ liệu realtime

- UI phải cập nhật dữ liệu trong < 1s

FR4: Điều khiển thiết bị

- Người dùng có thể bật/tắt LED
- Lệnh phải được gửi tới ESP32

FR5: Đồng bộ trạng thái

- UI phải phản ánh đúng trạng thái thiết bị
- Có trạng thái trung gian (waiting)

FR6: Lưu trữ dữ liệu

- Lưu lịch sử cảm biến vào database

FR7: Xem biểu đồ

- Hiển thị dữ liệu theo thời gian

4. Yêu cầu phi chức năng (Non-functional Requirements)

Mã yêu cầu	Loại yêu cầu	Mô tả chi tiết
NFR1	Hiệu năng (Performance)	Hệ thống phải đảm bảo độ trễ nhỏ hơn 1 giây và có khả năng xử lý tối thiểu 100 request/giây.
NFR2	Độ tin cậy (Reliability)	Hệ thống phải tự động kết nối lại MQTT khi mất kết nối và thực hiện retry khi xảy ra lỗi truyền dữ liệu.
NFR3	Khả năng mở rộng (Scalability)	Hệ thống phải cho phép bổ sung thêm thiết bị mới mà không ảnh hưởng đến hoạt động hiện tại.
NFR4	Bảo mật (Security)	Hệ thống phải hỗ trợ xác thực người dùng và mã hóa dữ liệu truyền tải bằng HTTPS.
NFR5	Khả dụng (Usability)	Giao diện người dùng phải thân thiện, dễ sử dụng và hỗ trợ responsive trên nhiều thiết bị.

5. Use Case Specification (Chi tiết)

UC1: Xem dữ liệu

- Actor: User
- Flow:

1. User mở web
2. Hệ thống hiển thị dữ liệu realtime

UC2: Điều khiển thiết bị

- Actor: Admin
- Flow:
    1. Click bật/tắt
    2. Gửi lệnh
    3. Nhận ACK
    4. Update UI

6. Yêu cầu giao diện (UI Requirements)

- Dashboard hiển thị:
    - Nhiệt độ
    - Độ ẩm
    - Ánh sáng
- Toggle switch điều khiển
- Biểu đồ lịch sử

7. Yêu cầu hệ thống (System Requirements)

Hardware

- ESP32
- Cảm biến DHT11, BH1750

Software

- Node.js
- Next.js
- MySQL

8. Ràng buộc hệ thống

- Phụ thuộc WiFi
- Phụ thuộc MQTT Broker

9. Tiêu chí chấp nhận (Acceptance Criteria)

- Dữ liệu hiển thị chính xác
- Điều khiển hoạt động ổn định
- Không mất đồng bộ trạng thái